Przystępne technologie sieciowe w embedded

Eryk Możdżeń

Koło Naukowe Robotyków KoNaR

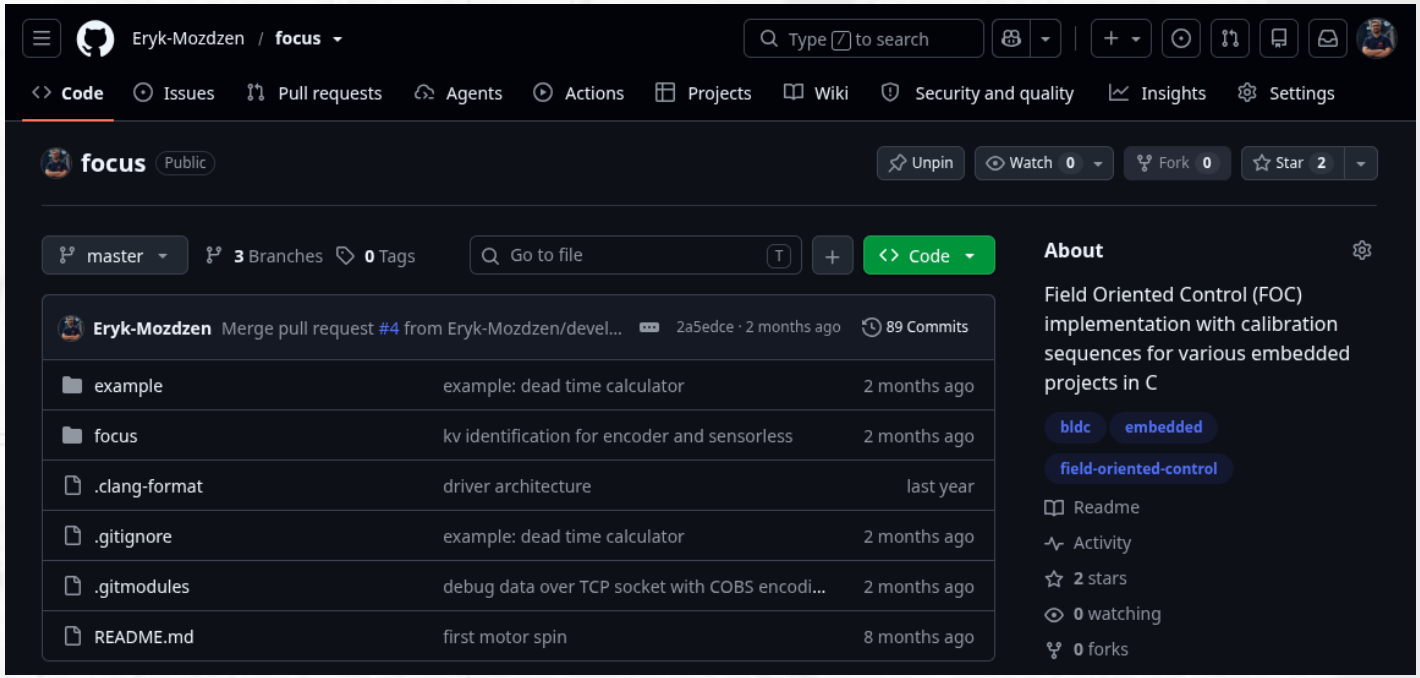
23.10.2026

Plan prezentacji

1. Wstęp
2. Biblioteka tinyUSB
3. Biblioteka lwIP
4. Serial Line Internet Protocol
5. Point-to-Point Protocol
6. Live demo
7. Podsumowanie

Motywacja

FOCUS - biblioteka do sterownia BLDC



- na płytce przewidziałem tylko USB do developmentu
- implementacja USB CDC od ST jest okropna
- zacząłem szukać alternatyw

tinyUSB

- aktywnie wspierana
- wspiera 67 rodzin procesorów różnych producentów
- implementuje stosy USB host/device
- implementuje różne "deskryptory"
- stosunkowo proste użycie
- example są, jest ich dużo, ale są mało czytelne

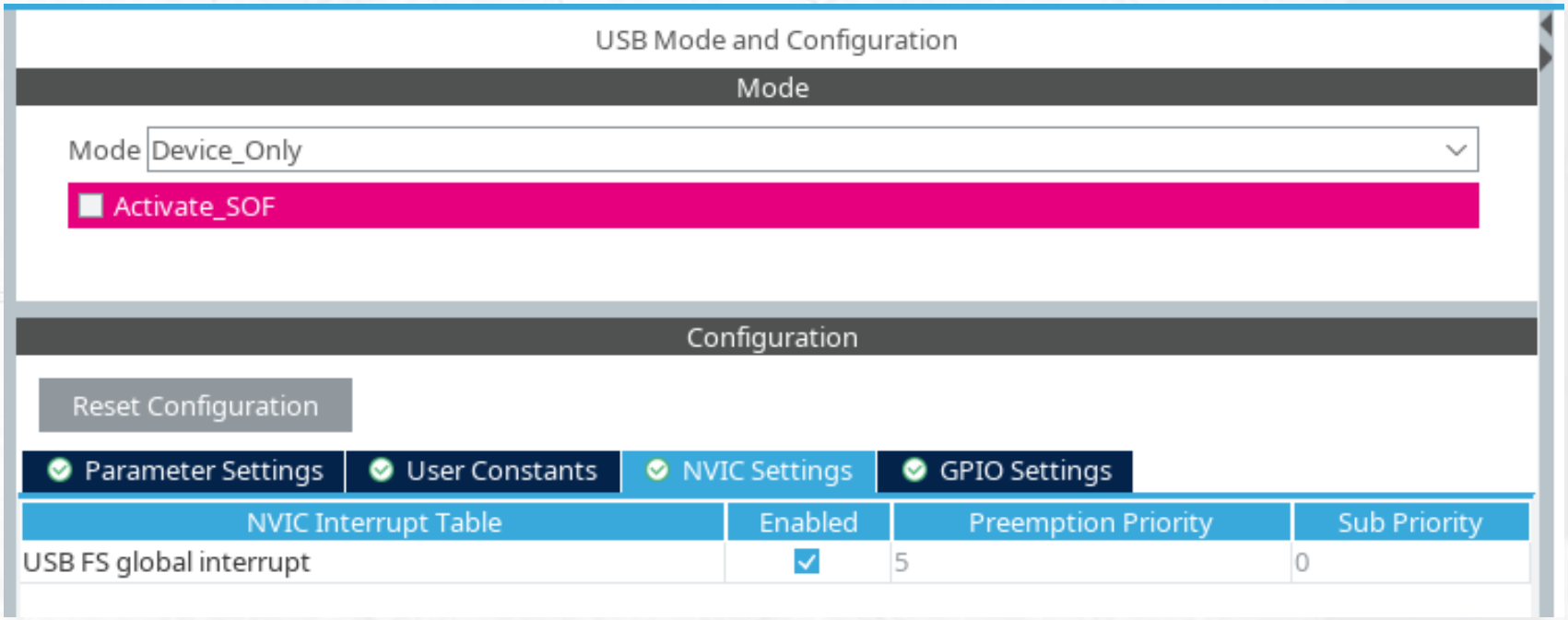


Przegląd możliwości

- Audio Class 1.0/2.0 (UAC1/UAC2)
- Bluetooth Host Controller Interface (BTH HCI)
- **Communication Device Class (CDC)**
- Device Firmware Update (DFU)
- Human Interface Device (HID)
- Printer class
- **Mass Storage Class (MSC)**
- Musical Instrument Digital Interface (MIDI)
- Media Transfer Protocol (MTP/PTP)
- **Network with RNDIS, Ethernet Control Model (ECM), Network Control Model (NCM)**
- Test and Measurement Class (USBTMC)

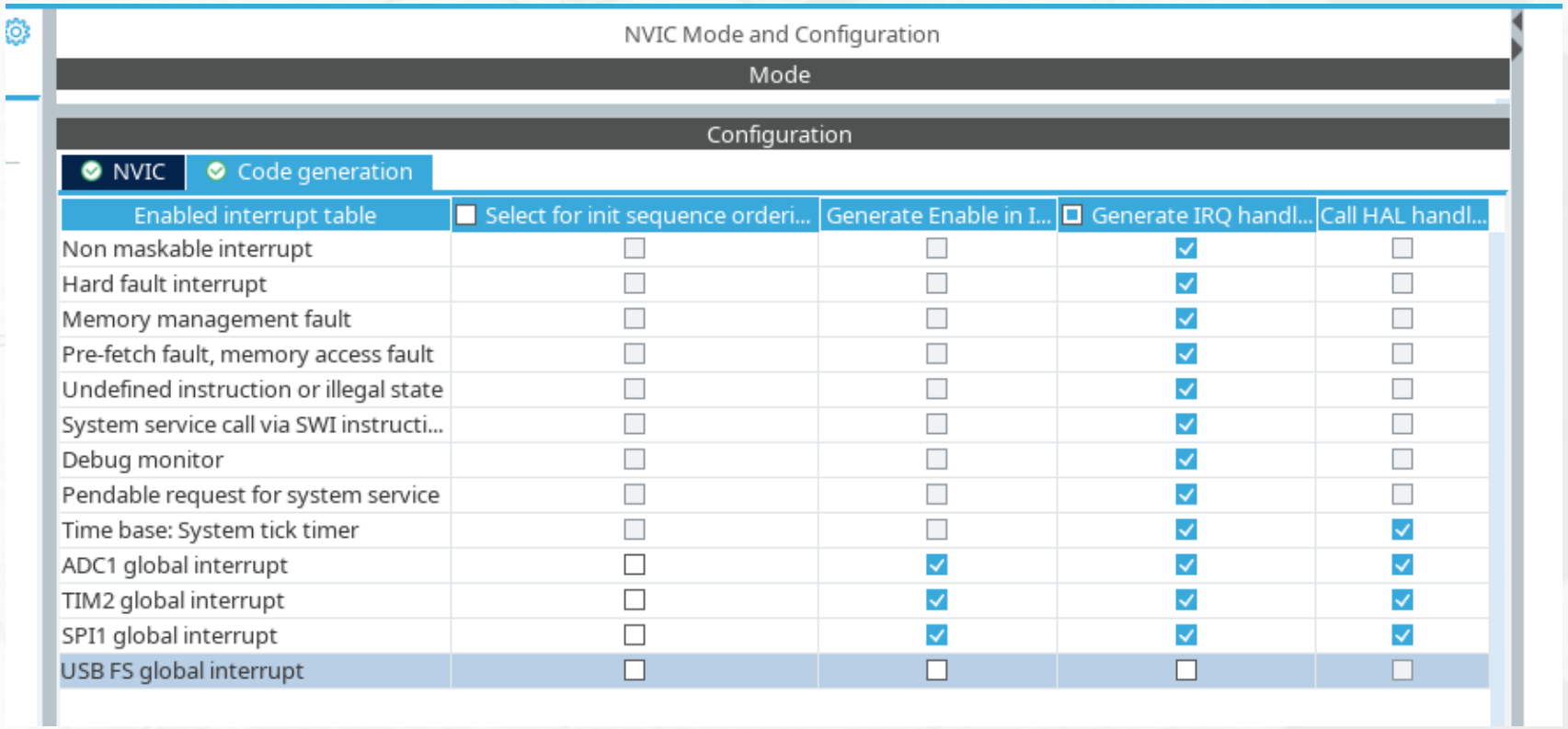
Użycie

CubeMX



Użycie

CubeMX c.d



NVIC Mode and Configuration

Mode

Configuration

NVIC | Code generation

Enabled interrupt table	Select for init sequence orderi...	Generate Enable in I...	Generate IRQ handl...	Call HAL handl...
Non maskable interrupt	☐	☐	☑	☐
Hard fault interrupt	☐	☐	☑	☐
Memory management fault	☐	☐	☑	☐
Pre-fetch fault, memory access fault	☐	☐	☑	☐
Undefined instruction or illegal state	☐	☐	☑	☐
System service call via SWI instructi...	☐	☐	☑	☐
Debug monitor	☐	☐	☑	☐
Pendable request for system service	☐	☐	☑	☐
Time base: System tick timer	☐	☐	☑	☑
ADC1 global interrupt	☐	☑	☑	☑
TIM2 global interrupt	☐	☑	☑	☑
SPI1 global interrupt	☐	☑	☑	☑
USB FS global interrupt	☐	☐	☐	☐

Użycie

kod

```c
void USB_DRD_FS_IRQHandler() {
    tud_int_handler(0);
}
```

```c
const tusb_rhport_init_t dev_init = {
    .role = TUSB_ROLE_DEVICE,
    .speed = TUSB_SPEED_FULL,
};

tusb_init(0, &dev_init);
```

Użycie

kod c.d.

Plik konfiguracyjny i deskryptory

- lekko modyfikowane względem exampli

Użycie klas urządzeń:

- kopiowane wprost z exampli
- biblioteka dostarcza dodatkowe pliki umożlwiające **integrację z lwIP**

Lightweight IP (lwIP)

- bogate community
- dużo kodu w internecie
- stworzona dla zastosowań embedded
- szerokie możliwości
- trudna w konfiguracji
- **słabo wspierana**

Możliwości

- IP (Internet Protocol, IPv4 and IPv6) **including packet forwarding**
- ICMP (Internet Control Message Protocol)
- IGMP (Internet Group Management Protocol)
- MLD (Multicast listener discovery for IPv6)
- ND (Neighbor discovery)
- DHCP, AutoIP/APIPA, ACD (Address Conflict Detection), DHCPv6
- UDP (User Datagram Protocol)
- TCP (Transmission Control Protocol)
- TLS
- **PPPoS and PPPoE (Point-to-point protocol over Serial/Ethernet)**
- DNS (Domain name resolver incl. mDNS)
- 6LoWPAN (via IEEE 802.15.4, BLE or ZEP)
- Optional Berkeley-like socket API

Możliwości c.d.

- **HTTP server with SSI and CGI**
- SNMPv2c agent with MIB compiler (Simple Network Management Protocol)
- SNTP (Simple network time protocol)
- NetBIOS name service responder
- MDNS (Multicast DNS) responder
- **iPerf server (and client) implementation**
- MQTT client (TLS support via altcp)

Serial Line Internet Protocol (SLIP)

- sposób na przesyłanie pakietów IP po łączu szeregowym
- w 1984 powstał
- w 1988 był powszechnie używany
- prosta serializacja/deserializacja
- brak sum kontrolnych
- brak mechanizmów retransmisji
- brak kompresji (potem dodali CSLIP)
- brak szyfrowania
- współcześnie wyparty przez PPP ale dalej wspierany

```
1 sudo slattach -L -p slip -s 115200 /dev/ttyUSB0
2 sudo ip addr add 192.168.9.2/24 dev sl0
3 sudo ip link set sl0 up
```

Point-to-Point Protocol (PPP)

- powstał w 1994
- auto-negocjacja prędkości łacza, wielkosci buforów itp
- prosta serializacja/deserializacja
- wykrywanie błędów i retransmisja
- mechanizmy kompresji i szyfrowania
- PPP over Serial (PPPoS)
- PPP over Ethernet (PPPoE)
- PPP over ATM (PPPoA, PPPoEoA)
- współcześnie używany przez modemy GSM (AT+PPP) oraz dostawców internetu typu Orange

Point-to-Point Protocol (PPP)

c.d.

- urządzenie "server" czeka na rozpoczęcie sesji
- urządzenie "client" rozpoczyna sesję
- negocjacja parametrów łącza
- urządzenie "server" ustala IP po obu stronach
- podsieć gotowa do użytku

```
1 sudo pppd /dev/ttyUSB0 921600 192.168.7.1:192.168.7.2
    local noauth debug nodetach nocrtscts
```

Sterownik silnika

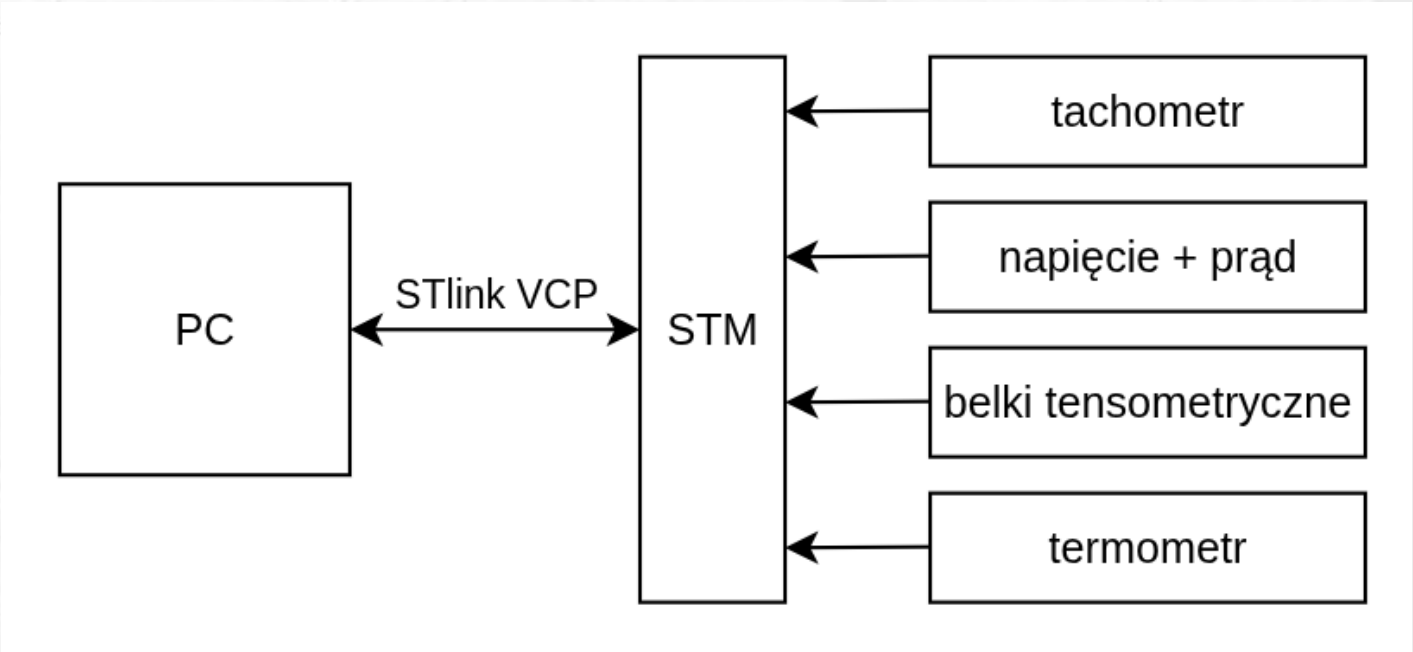
TinyUSB (RNDIS/ECM/NCM + DHCP) + lwIP

- STM32H5
- warstwa aplikacji: streamy UDP + telnet,
- połączenie plug & play

Wizualizacja danych

VCOM + lwIP (PPP)

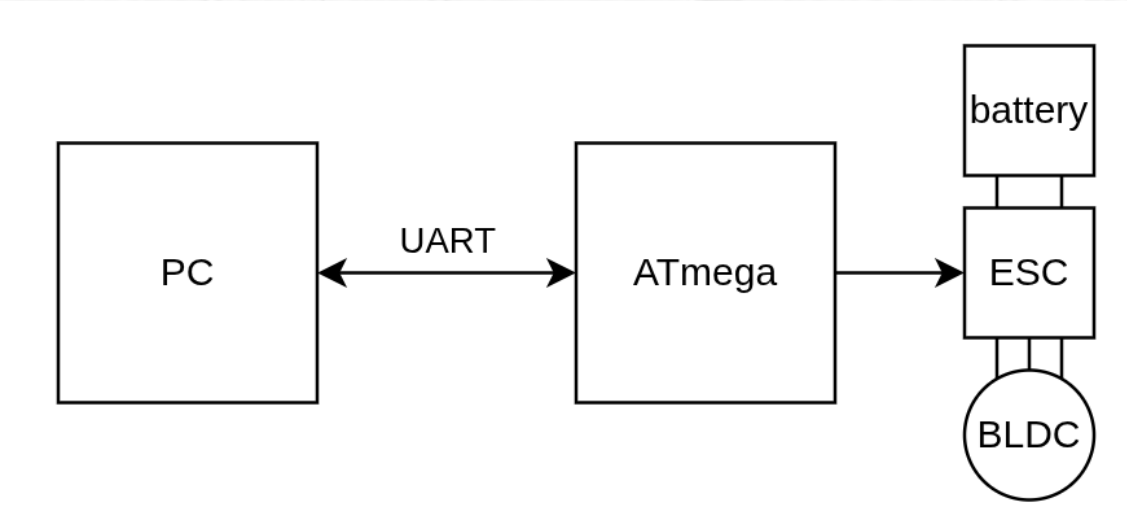
- STM32U0
- warstwa aplikacji: serwer HTTP + websocket
- łatwa integracja UI z urządzeniem
- lwIP nie ma wsparcia dla websocketów
  (sam pisałem, może znajdą się na to patche)
- tutaj prawdopodobnie lepiej spisze się Mongoose



Zadajnik PWM

uIP (SLIP)

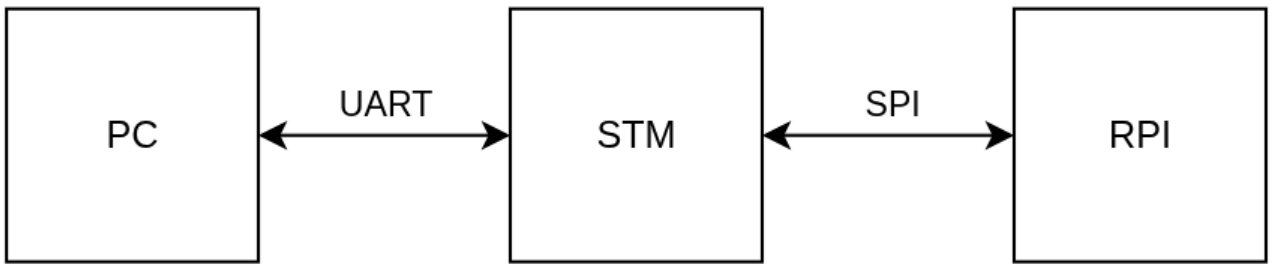
- ATmega32A (2kB RAM, 32kB FLASH)
- za mało RAMu na lwIP, więc użyłem uIP (2003)
- za mało RAMu na PPP, więc użyłem SLIP
- warstwa aplikacji: telnet
- nie ma sensu ale udowadnia że się da i działa
- **jeśli to działa, to na innych zabytkach pewnie też**



Bieda router

lwIP (PPP + ip forwarding)

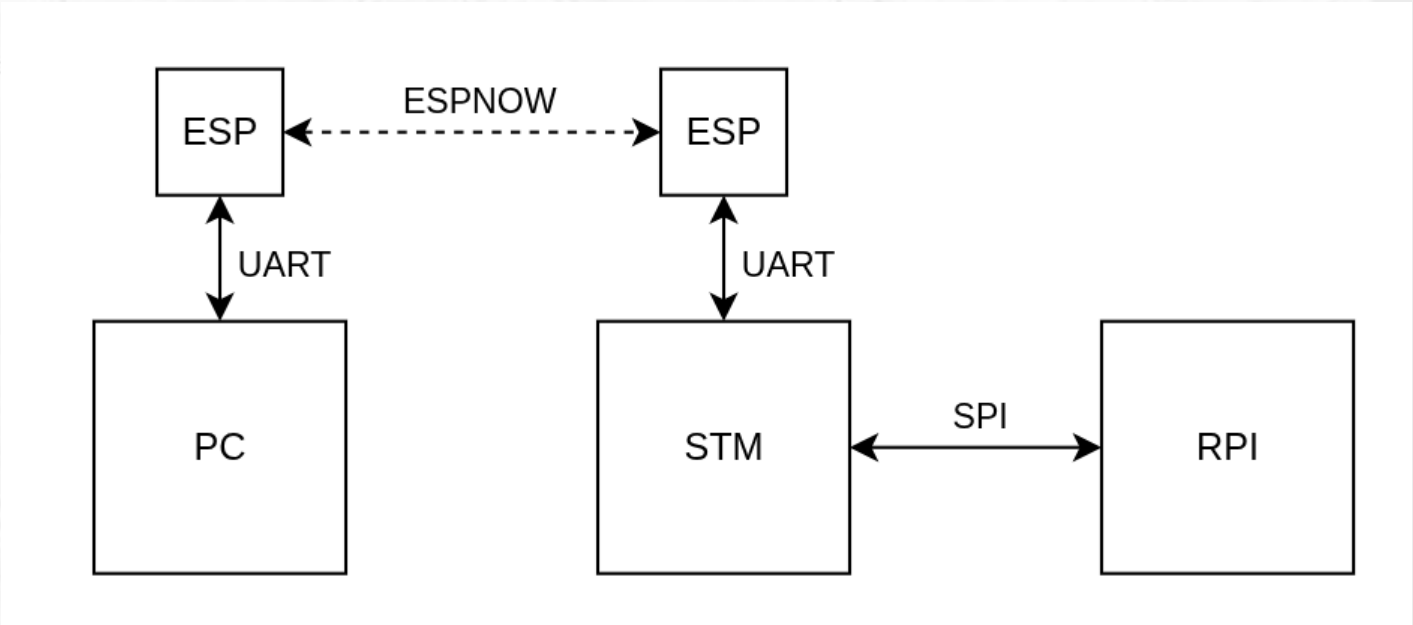
- STM32F4
- interfejsy PPP (2x UART + 1x SPI)
- włączona opcja "IP forwarding"
- umożliwia komunikację PC-STM
- umożlwiia komunikację STM-RPI
- **umożliwia komunikację PC-RPI**
- lwIP wymaga uwagi (patche)



Bieda router dla UAV

lwIP (PPP + ip forwarding)

- PPP działa dobrze również **bezprzewodowo**
- rodzaj radia nie ma znaczenia, byle dwukierunkowe (duplexowe)
- przepustowość UARTa nie jest górnym ograniczeniem
- przepustowość radia jest górnym ograniczeniem



Zady i walety komunikacji sieciowej

Zalety:

- wsparcie na każdym systemie operacyjnym
- łatwa integracja z istniejącym oprogramowaniem
- umożliwia używanie dobrodziejstw sieciówki: routing, usługi, narzędzia
- **architektura fizyczna przestaje implikować architekturę logiczną**

```
async def main():
    async with (
        Focus("192.168.8.1") as focus,
        ESC("192.168.9.1") as esc,
        Bench("ws://192.168.7.2:81/data") as bench,
    ):
        bus = Bus()
```

Wady:

- wymaga dużo RAMu
- nie jest hard real-time (czekam na TSN)
- wymaga wiedzy z sieciówki (okazja do nauki)